

HANDS-ON WORKBOOK

PostgreSQL High Availability Cluster

Docker · Kubernetes · Namespace **postgres-ha** · Cluster **acid-postgres-ha**
Patroni · Zalando Postgres Operator (acid.zalan.do) · Troubleshooting Labs

Step-by-step deployment, verification, failover drills, and incident exercises for operators who run Patroni-managed PostgreSQL on containers and Kubernetes.

Item	Value
Target cluster name	acid-postgres-ha
Kubernetes namespace	postgres-ha
Operator CRD	postgresql.acid.zalan.do / v1
HA engine	Patroni + Spilo image
DCS on Kubernetes	Kubernetes Endpoints / ConfigMaps (Patroni K8s DCS)
DCS on Docker lab	etcd v3
Suggested PG version	16
Instance count (lab)	3 (1 primary + 2 replicas)

1. Purpose and how to use this document

This workbook walks a complete path from a local Docker Patroni cluster to a production-shaped Kubernetes deployment using the Zalando Postgres Operator. The Kubernetes cluster name **acid-postgres-ha** in namespace **postgres-ha** matches the operator convention: the prefix **acid** is the teamId, and the custom resource kind is **postgresql** in API group **acid.zalan.do**.

Work the sections in order the first time. Later, jump to Part 6 when an incident appears. Each troubleshooting exercise states the symptom, how to inject the failure safely in a lab, the diagnostic commands, expected findings, and the recovery path.

Safety

Do not run destructive exercises (pod delete, PVC wipe, etcd loss, forced failover) against a production cluster. Use a dedicated lab cluster or minikube/kind/k3s. Always confirm you are in namespace postgres-ha before deleting objects.

2. Architecture

2.1 Components

Patroni is a Python HA controller that wraps PostgreSQL. It performs leader election in a Distributed Configuration Store (DCS), manages streaming replication, runs health checks on REST port 8008, and executes automatic failover when the primary lease expires. On Kubernetes the Zalando operator (Spilo container) embeds Patroni, PostgreSQL,

WAL-G, and helper scripts. The operator watches Postgresql CRs and reconciles StatefulSets, Services, Endpoints, Secrets, and PodDisruptionBudgets.

Layer	Docker lab	Kubernetes (this workbook)
PostgreSQL + Patroni	3 containers (patroni1–3)	StatefulSet acid-postgres-ha-0/1/2
DCS / consensus	3-node etcd	Kubernetes API (Endpoints + ConfigMaps)
Write VIP / router	HAProxy :5000 → primary	Service acid-postgres-ha (master)
Read router	HAProxy :5001 → replicas	Service acid-postgres-ha-repl
Replica identity	PATRONI_NAME	Pod name = Patroni member name
Credentials	env / patroni.yml	K8s Secrets (postgres.*, standby.*)
Controller	you / compose	postgres-operator Deployment

2.2 Traffic and roles

- **Primary (leader)** accepts writes. Only one member holds the Patroni leader lock.
- **Replicas (replicas / sync / async)** stream WAL from the primary. They serve reads.
- **Master Service** always points at the current leader (operator updates Endpoints).
- **Replica Service** load-balances ready standbys.
- Clients should connect to the Service DNS, never to a pod IP, so failover is transparent.

2.3 Naming map for this lab

```

Namespace:      postgres-ha
Postgresql CR:  acid-postgres-ha
teamId:         acid
StatefulSet / pods: acid-postgres-ha-0, acid-postgres-ha-1, acid-postgres-ha-2
Master Service: acid-postgres-ha.postgres-ha.svc.cluster.local:5432
Replica Service: acid-postgres-ha-repl.postgres-ha.svc.cluster.local:5432
Patroni REST:   http://<pod-ip>:8008/patroni (or kubectl port-forward)
Superuser secret: postgres.acid-postgres-ha.credentials.postgresql.acid.zalan.do
Operator:       typically deployed in postgres-operator or postgres-ha

```

3. Prerequisites

Tool	Minimum	Check
Docker Engine + Compose	24+ / Compose v2	docker version && docker compose version
kubectl	1.27+	kubectl version --client
A Kubernetes cluster	kind / k3s / EKS / GKE	kubectl get nodes
Helm (recommended)	3.12+	helm version
psql client	14+	psql --version
curl / jq	any	jq --version
StorageClass	dynamic PV provisioner	kubectl get sc

On Kubernetes you need a default or named StorageClass that can provision a PVC per instance (3 × volume.size). For kind, install a local-path or Rancher provisioner. The operator needs permission to create Endpoints, Pods, StatefulSets, PDBs, and Secrets in the watched namespace.

4. Part A — Patroni cluster on Docker

Use this part to learn Patroni itself without Kubernetes. Official Patroni images are intended for development. Build from the Patroni repository or use a known compose stack with etcd + 3 Patroni nodes + HAProxy.

4.1 Clone and start the official demo stack

```
git clone https://github.com/patroni/patroni.git
cd patroni
docker build -t patroni .
docker compose up -d
docker compose ps
```

Expected containers: demo-etcd1/2/3, demo-patroni1/2/3, demo-haproxy. HAProxy publishes host port 5000 (writes → primary) and 5001 (reads → replicas).

4.2 Inspect cluster state

```
# From a Patroni container
docker exec -it demo-patroni1 patronictl -c /home/postgres/postgres.yml list

# REST health of a member
docker exec -it demo-patroni1 curl -s http://localhost:8008/patroni | jq .

# Connect through HAProxy write port
psql "host=127.0.0.1 port=5000 user=postgres password=postgres dbname=postgres" -c "SELECT pg_is_in_recovery();"
# Expect: f (false = primary)

psql "host=127.0.0.1 port=5001 user=postgres password=postgres dbname=postgres" -c "SELECT pg_is_in_recovery();"
# Expect: t (true = replica)
```

4.3 Docker failover drill

```
# Identify the current leader from patronictl list
docker stop demo-patroni1          # if it was the leader
sleep 15
docker exec -it demo-patroni2 patronictl -c /home/postgres/postgres.yml list
# A former replica should now show Leader / running
psql "host=127.0.0.1 port=5000 ..." -c "SELECT inet_server_addr(), pg_is_in_recovery();"

# Restore the old node; it should rejoin as replica
docker start demo-patroni1
docker exec -it demo-patroni2 patronictl -c /home/postgres/postgres.yml list
```

What you should internalize

Failover time is bounded by `ttl + loop_wait + detection`. HAProxy health-checks Patroni REST so port 5000 moves with the leader. A restarted node must not become primary if it is behind (`maximum_lag_on_failover`).

5. Part B — Kubernetes: namespace postgres-ha, cluster acid-postgres-ha

5.1 Create the namespace

```
kubectl create namespace postgres-ha
kubectl config set-context --current --namespace=postgres-ha
kubectl get ns postgres-ha
```

5.2 Install the Zalando Postgres Operator (Helm)

Install the operator in its own namespace so RBAC stays isolated. Watch postgres-ha (and optionally others) via Helm values.

```

helm repo add postgres-operator-charts \
  https://opensource.zalando.com/postgres-operator/charts/postgres-operator
helm repo update

kubectl create namespace postgres-operator

# Watch only the lab namespace
cat > /tmp/po-values.yaml <<'EOF'
configKubernetes:
  watched_namespace: "postgres-ha"
configPostgresPodResources:
  default_cpu_request: 100m
  default_memory_request: 200Mi
  default_cpu_limit: "1"
  default_memory_limit: 500Mi
EOF

helm install postgres-operator postgres-operator-charts/postgres-operator \
  --namespace postgres-operator \
  --values /tmp/po-values.yaml

kubectl -n postgres-operator rollout status deploy/postgres-operator
kubectl -n postgres-operator logs -l name=postgres-operator --tail=50

```

5.3 Manual install alternative (manifests)

```

git clone https://github.com/zalando/postgres-operator.git
cd postgres-operator
# Edit manifests so ServiceAccount / Deployment namespaces match your choice,
# and set WATCHED_NAMESPACE=postgres-ha on the operator Deployment.
kubectl apply -f manifests/configmap.yaml
kubectl apply -f manifests/operator-service-account-rbac.yaml
kubectl apply -f manifests/postgres-operator.yaml
kubectl apply -f manifests/api-service.yaml

```

5.4 Apply the Postgresql custom resource

```
cat <<'EOF' | kubectl apply -n postgres-ha -f -
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-postgres-ha
  namespace: postgres-ha
  labels:
    team: acid
    purpose: lab
spec:
  teamId: "acid"
  numberOfInstances: 3
  volume:
    size: 5Gi
    # storageClass: standard    # set if you do not have a default SC
  users:
    appuser:
      - createdb
    labadmin:
      - superuser
      - createdb
  databases:
    appdb: appuser
  postgresql:
    version: "16"
    parameters:
      shared_buffers: "128MB"
      max_connections: "100"
      log_min_duration_statement: "1000"
      idle_in_transaction_session_timeout: "60000"
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: "1"
      memory: 512Mi
  patroni:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 33554432
    enableConnectionPooler: false
    enableReplicaConnectionPooler: false
EOF
```

5.5 Watch the operator reconcile

```
# Cluster object status
kubectl -n postgres-ha get postgresql acid-postgres-ha -w
# Columns typically: NAME TEAM VERSION PODS VOLUME CPU-REQ MEMORY-REQ AGE STATUS

kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha -o wide
kubectl -n postgres-ha get sts,svc,ep,pdb,secret | grep acid-postgres-ha
kubectl -n postgres-ha describe postgresql acid-postgres-ha

# Operator view of events
kubectl -n postgres-operator logs -l name=postgres-operator --tail=100
```

Healthy end state: Postgresql STATUS = Running, three pods Ready, one pod role=master (label spilo-role=master), two spilo-role=replica. Services acid-postgres-ha and acid-postgres-ha-repl exist. Endpoints of the master Service contain only the leader pod IP.

5.6 Get credentials and connect

```
# Superuser password (username is postgres)
kubectl -n postgres-ha get secret postgres-acid-postgres-ha-credentials.postgresql.acid.zalan.do \
  -o jsonpath='{.data.password}' | base64 -d; echo

# Application user
kubectl -n postgres-ha get secret appuser-acid-postgres-ha-credentials.postgresql.acid.zalan.do \
  -o jsonpath='{.data.username}' | base64 -d; echo
kubectl -n postgres-ha get secret appuser-acid-postgres-ha-credentials.postgresql.acid.zalan.do \
  -o jsonpath='{.data.password}' | base64 -d; echo

# Port-forward the master Service
kubectl -n postgres-ha port-forward svc/acid-postgres-ha 15432:5432
psql "host=127.0.0.1 port=15432 user=postgres dbname=postgres sslmode=require"

# Or exec into the current master
MASTER=$(kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha,spilo-role=master -o jsonpath='{.items[0].metadata.name}')
kubectl -n postgres-ha exec -it "$MASTER" -- bash
# inside the pod
patronictl list
psql -U postgres -c "SELECT application_name, client_addr, state, sync_state FROM pg_stat_replication;"
curl -s localhost:8008/patroni | jq .
```

5.7 Patroni from inside Spilo

```
# Useful Patroni / Spilo paths
# Config:      /home/postgres/postgres.yml (generated)
# PGDATA:      /home/postgres/pgdata/pgroot/data
# Patroni log: process stdout (kubectl logs)
# REST:        :8008 /health /primary /replica /patroni /cluster /config /failover

kubectl -n postgres-ha exec -it acid-postgres-ha-0 -- patronictl list
kubectl -n postgres-ha exec -it acid-postgres-ha-0 -- curl -s http://localhost:8008/cluster | jq .
```

5.8 Scale, switchover, and planned maintenance

```
# Scale replicas by editing the CR (operator rolls the StatefulSet)
kubectl -n postgres-ha patch postgresql acid-postgres-ha --type merge -p '{"spec":{"numberOfInstances":3}}'

# Planned switchover (NOT failover). Moves primary to a named healthy replica.
kubectl -n postgres-ha exec -it acid-postgres-ha-0 -- \
  patronictl switchover --candidate acid-postgres-ha-1 --force

# Confirm Service endpoints followed the new leader
kubectl -n postgres-ha get ep acid-postgres-ha -o yaml
kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha --show-labels
```

6. How data, WAL, backups and streaming actually live

Most production incidents are not “Patroni forgot the leader”. They are WAL that cannot be archived, a PVC that filled up, a backup that never landed in the bucket, a clone that bootstrapped from the wrong prefix, or a replica that stopped streaming after a timeline change. This section is the map; the exercises that follow force you to use it.

6.1 On-disk layout inside a Spilo pod

```
# PVC is mounted at /home/postgres/pgdata (claim name pgdata-acid-postgres-ha-N)
/home/postgres/pgdata/pgroot/
  data/          # PGDATA – catalog, heap, WAL (pg_wal/), replication slots
  pg_log/        # PostgreSQL / archive / restore command logs
  pg_wal/        # sometimes a symlink target; confirm with `ls -l`

# Patroni generated config
/home/postgres/postgres.yml

# WAL-G / WAL-E environment directories (created by Spilo)
/run/etc/wal-e.d/env          # live archive + backup destination
/run/etc/wal-e.d/env-clone-* # source bucket used ONLY during clone/bootstrap
```

What	Where it is stored	What fails if it breaks
User tables / indexes	PVC pgdata-*-N under PGDATA	Pod crash, corrupt relation, full disk
pg_wal (WAL segments)	PGDATA/pg_wal on the same PVC	Primary stalls; replicas disconnect; disk fills
Replication slots	PGDATA/pg_replslot	WAL never recycled if a slot is stale
Base backups	Object storage (S3/GCS/Azure) via WAL-G	Cannot clone or PITR
Archived WAL	Same bucket, wal_prefix/.../wal_005/	PITR stops at last archived segment
Logical dump (optional)	spec.enableLogicalBackup + S3	Schema-only recovery, not HA restore

6.2 Enable physical backups (WAL-G) for the lab

Spilo archives WAL with **archive_command** = envdir WAL-G wal-push, and takes a nightly base backup (BACKUP_SCHEDULE, default 01:00 UTC). The operator does not invent a bucket; you must give every pod the WAL-G environment, usually via pod_environment_configmap or env on the Postgres CR.

```
# Operator-level ConfigMap (applies to all clusters the operator builds)
apiVersion: v1
kind: ConfigMap
metadata:
  name: pod-env-overrides
  namespace: postgres-operator
data:
  USE_WALG_BACKUP: "true"
  USE_WALG_RESTORE: "true"
  CLONE_USE_WALG_RESTORE: "true"
  AWS_ENDPOINT: "http://minio.minio.svc:9000"
  AWS_REGION: "us-east-1"
  AWS_ACCESS_KEY_ID: "minio"
  AWS_SECRET_ACCESS_KEY: "minio123"
  AWS_S3_FORCE_PATH_STYLE: "true"
  WAL_S3_BUCKET: "pg-backups"
  WALG_S3_PREFIX: "s3://pg-backups/spilo/acid-postgres-ha"
  BACKUP_SCHEDULE: "*/15 * * * *" # lab: every 15 min, not nightly
  BACKUP_NUM_TO_RETAIN: "3"
  WALG_DOWNLOAD_CONCURRENCY: "2"

# Point the operator at it (Helm values / OperatorConfiguration)
# configKubernetes.pod_environment_configmap: postgres-operator/pod-env-overrides

# Per-cluster override (acid.zalan.do CR)
# spec.env:
#   - name: WAL_S3_BUCKET
#     value: pg-backups
```

6.3 Manual backup, list, and archive check

```

MASTER=$(kubectl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master \
-o jsonpath='{.items[0].metadata.name}')

# Settings Postgres is actually using
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW archive_mode;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW archive_command;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW restore_command;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SELECT * FROM pg_stat_archiver;"

# Force a WAL switch so archive_command runs
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SELECT pg_switch_wal();"

# List base backups in the bucket
kubectl -n postgres-ha exec "$MASTER" -- \
  envdir /run/etc/wal-e.d/env wal-g backup-list

# Take an out-of-band base backup NOW
kubectl -n postgres-ha exec "$MASTER" -- \
  envdir /run/etc/wal-e.d/env /scripts/postgres_backup.sh /home/postgres/pgdata/pgroot/data

# Archive / restore helper logs
kubectl -n postgres-ha exec "$MASTER" -- ls -lt /home/postgres/pgdata/pgroot/pg_log | head

```

6.4 Restore / clone patterns (never overwrite prod in place first)

```

# A. Clone a LIVE cluster (pg_basebackup via Patroni) into a new CR
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-postgres-ha-clone
  namespace: postgres-ha
spec:
  teamId: acid
  numberOfInstances: 2
  volume: { size: 5Gi }
  postgresql: { version: "16" }
  clone:
    cluster: acid-postgres-ha      # source in the same namespace

# B. PITR from object storage (WAL-G). timestamp is RFC3339 with timezone.
#   uid = metadata.uid of the SOURCE Postgresql CR at backup time.
spec:
  clone:
    cluster: acid-postgres-ha
    uid: "11111111-2222-3333-4444-555555555555"
    timestamp: "2026-09-29T00:30:00+00:00"
# Pods must have CLONE_* env (CLONE_WAL_S3_BUCKET, CLONE_USE_WALG_RESTORE=true, keys...)

# C. In-place restore of the SAME name is a last resort: delete the CR (and PVCs),
#   recreate with spec.clone pointing at the old uid + timestamp. Test on a clone CR first.

```

Storage rule

Replicas are not backups. A streaming replica shares the same failure domain as the primary (bad migration, DROP TABLE, ransomware-style UPDATE). WAL-G basebackup + continuous wal-push is the backup. PVCs are the running copy. Logical dumps are for cross-version or subset recovery, not for RTO of an HA cluster.

7. Troubleshooting exercises

Work these in a lab. For each exercise: record the time, the command output that proved the diagnosis, and the command that restored service. Do not skip the verification step — a cluster that looks Running can still have a stale master Service, WAL that is not archiving, or a replica that is not streaming.

Exercise 1 — Cluster CR exists but no pods

Symptom. `kubectl get postgresql` shows `CreateFailed / Invalid`, or stays empty with 0 pods.

Diagnose.

```
# Inject: apply a bad manifest (invalid PG version or missing StorageClass)
# Diagnose
kubectl -n postgres-ha get postgresql acid-postgres-ha -o yaml | sed -n '/status:$/, $p'
kubectl -n postgres-ha describe postgresql acid-postgres-ha
kubectl -n postgres-operator logs -l name=postgres-operator --tail=200
kubectl get sc
kubectl -n postgres-ha get events --sort-by=.lastTimestamp | tail
```

Resolve and verify. Operator rejected the CR (unsupported version, unknown parameter) or PVCs are Pending because no StorageClass matches. Fix the spec, set `spec.volume.storageClass`, or install a provisioner. Re-apply the CR. Confirm pods `Scheduling` → `Running`.

Exercise 2 — PVC Pending / pod Pending

Symptom. Pods stuck Pending. Events: `FailedScheduling`, unbound `PersistentVolumeClaims`.

Diagnose.

```
kubectl -n postgres-ha get pvc
kubectl -n postgres-ha describe pvc pgdata-acid-postgres-ha-0
kubectl -n postgres-ha describe pod acid-postgres-ha-0 | tail -40
kubectl get sc, pv
```

Resolve and verify. Wrong `storageClass` name, quota, or zone mismatch. Patch the CR `storageClass`, delete the unbound PVC only if the pod is not running (data loss risk), let the operator recreate. On kind use `rancher.io/local-path` or similar.

Exercise 3 — CrashLoopBackOff on a Spilo pod

Symptom. One or more `acid-postgres-ha-N` pods restart. Ready 0/1.

Diagnose.

```
kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha
kubectl -n postgres-ha logs acid-postgres-ha-1 --tail=150
kubectl -n postgres-ha logs acid-postgres-ha-1 --previous --tail=80
kubectl -n postgres-ha describe pod acid-postgres-ha-1
# Common log signatures:
#   FATAL:  lock file "postmaster.pid" already exists
#   FATAL:  data directory has wrong ownership
#   patroni.exceptions.PatroniFatalException: Can not find proper pg_rewrite
#   DCSError / 403 Forbidden talking to Kubernetes API
```

Resolve and verify. Read the first FATAL. Ownership issues mean the PVC was reused with a different `fsGroup`. DCS 403 means the pod `ServiceAccount` cannot patch `Endpoints` — fix operator RBAC. If PGDATA is corrupt, remove that instance (see Exercise 8) rather than deleting all PVCs.

Exercise 4 — Split-brain fear: two pods claim master

Symptom. Two pods have label `spilo-role=master`, or two IPs on the master Service.

Diagnose.

```
kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha --show-labels
kubectl -n postgres-ha get ep acid-postgres-ha -o yaml
for p in 0 1 2; do
  echo "=== acid-postgres-ha-$p ==="
  kubectl -n postgres-ha exec acid-postgres-ha-$p -- curl -s http://localhost:8008/patroni
done
kubectl -n postgres-ha exec acid-postgres-ha-0 -- patronictl list
```

Resolve and verify. Patroni + K8s DCS must elect exactly one leader. If labels lie, wait one ttl (30s) and re-check; the operator refreshes labels. True dual-primary is rare and usually means two independent clusters sharing a name or a broken DCS. Do not write to both. Isolate extras (see Exercise 9) and keep the member that has the newest timeline / highest LSN.

Exercise 5 — Unplanned failover (kill the primary pod)

Symptom. Writes hang or fail for several seconds; then a replica is promoted.

Diagnose.

```
# Inject
MASTER=$(kubectl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master -o name)
echo "Killing $MASTER"
kubectl -n postgres-ha delete $MASTER --force --grace-period=0

# Diagnose during the event
watch -n 2 'kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha --show-labels
kubectl -n postgres-ha get ep acid-postgres-ha
kubectl -n postgres-ha exec acid-postgres-ha-1 -- patronictl list 2>/dev/null'

# Verify application path
kubectl -n postgres-ha port-forward svc/acid-postgres-ha 15432:5432
# other terminal
psql "host=127.0.0.1 port=15432 user=postgres dbname=appdb" \
  -c "CREATE TABLE IF NOT EXISTS failover_lab(ts timestamptz default now()); INSERT INTO failover_lab DEFAULT VALUES; SELECT ts;"
```

Resolve and verify. RTO $\approx$ ttl (30s) + detection. The deleted pod is recreated by the StatefulSet and must rejoin as replica (rewind or basebackup). Confirm `pg_stat_replication` on the new primary shows the returning member in state streaming.

Exercise 6 — Replica lag / not streaming

Symptom. `patronictl list` shows Replica / stopped or streaming lag growing. Reads are stale.

Diagnose.

```
kubectl -n postgres-ha exec acid-postgres-ha-0 -- patronictl list
# On current master
kubectl -n postgres-ha exec -it $(kubectl -n postgres-ha get pod -l spilo-role=master -o name) -- \
  psql -U postgres -c "SELECT pid, application_name, client_addr, state, sent_lsn, replay_lsn,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
    FROM pg_stat_replication;"
kubectl -n postgres-ha logs acid-postgres-ha-2 --tail=80
```

Resolve and verify. Causes: network NetworkPolicy, disk full on replica PVC, rewind failure after timeline change, or `wal_keep_size` too small so WAL was recycled. Free disk, reclone the replica (`patronictl reinit --force`) only after confirming it is not the leader.

Exercise 7 — Disk full

Symptom. Primary logs PANIC could not write lock file / No space left on device. Failover may loop.

Diagnose.

```
kubectl -n postgres-ha exec acid-postgres-ha-0 -- df -h /home/postgres/pgdata
kubectl -n postgres-ha exec acid-postgres-ha-0 -- du -sh /home/postgres/pgdata/pgroot/data/*
# Inject in lab only: fallocate a huge dummy file inside PGDATA (do not do this in prod)
```

Resolve and verify. Zalando operator supports volume resize if the StorageClass allows expansion: raise spec.volume.size and wait for the PVC to grow (often without restart on CSI drivers that support it). Then VACUUM / drop leftover files. If WAL filled the disk, check archive_command / WAL-G and replication slots holding WAL.

Exercise 8 — Reinitialize a broken replica

Symptom. One member is Stopped / Timeline mismatch / rewind failed.

Diagnose.

```
# Confirm it is NOT the leader
kubectl -n postgres-ha exec acid-postgres-ha-0 -- patronictl list
# Reinit wipes local PGDATA and basebackups from the leader
kubectl -n postgres-ha exec -it acid-postgres-ha-0 -- \
    patronictl reinit acid-postgres-ha acid-postgres-ha-2 --force
kubectl -n postgres-ha logs -f acid-postgres-ha-2
```

Resolve and verify. reinit is the standard repair for a diverged replica. Watch for basebackup completed and state running. If reinit loops, delete the pod's PVC and the pod so StatefulSet recreates both (data on that replica only is lost).

Exercise 9 — Operator not reconciling (stale cluster)

Symptom. You patch numberOfInstances or parameters; nothing happens.

Diagnose.

```
kubectl -n postgres-operator get deploy postgres-operator
kubectl -n postgres-operator logs -l name=postgres-operator --tail=200
kubectl -n postgres-ha get postgresql acid-postgres-ha -o jsonpath='{.metadata.annotations}' ; echo
# Check watched namespace
kubectl -n postgres-operator get deploy postgres-operator -o yaml | grep -i watch
kubectl get crd postgresqls.acid.zalan.do
```

Resolve and verify. Operator pod down, watching the wrong namespace, or another operator instance owns the cluster via annotation acid.zalan.do/controller. Restart the operator Deployment. Confirm the CRD is installed. Avoid two operators watching the same ns.

Exercise 10 — Authentication failures from the app

Symptom. psql / app: password authentication failed for user appuser.

Diagnose.

```
kubectl -n postgres-ha get secret | grep acid-postgres-ha
kubectl -n postgres-ha get secret appuser.acid-postgres-ha.credentials.postgresql.acid.zalan.do -o yaml
# Compare username/password the app uses vs Secret data (base64 -d)
# Inside PG
psql -U postgres -c "\du+"
psql -U postgres -c "SELECT rolname, rolcanlogin FROM pg_roles;"
```

Resolve and verify. The operator generates Secrets and rotates them when you change users in the CR. Apps must mount the Secret or pull it at deploy time. Custom pg_hba in spec.patroni.pg_hba can block CIDRs. Connection string must target the Service, port 5432, and often sslmode=require (Spilo enables TLS).

Exercise 11 — NetworkPolicy or DNS isolates Patroni

Symptom. Members cannot see each other; patronictl list shows isolated nodes; failover flaps.

Diagnose.

```
kubectctl -n postgres-ha get networkpolicy
kubectctl -n postgres-ha exec acid-postgres-ha-0 -- curl -sv http://acid-postgres-ha-1.acid-postgres-ha.postgres-ha.svc:8008
kubectctl -n postgres-ha exec acid-postgres-ha-0 -- getent hosts acid-postgres-ha-1.acid-postgres-ha
# Kubernetes DCS needs talk to kubernetes.default.svc:443 as well
```

Resolve and verify. Allow pods labeled cluster-name=acid-postgres-ha to reach each other on 5432 and 8008 and to reach the Kubernetes API. Headless Service acid-postgres-ha provides stable DNS for members. Broken CoreDNS will look like a Patroni bug.

Exercise 12 — Controlled restore from a bad migration

Symptom. Schema change broke the app. Cluster is otherwise healthy.

Diagnose.

```
# This operator uses WAL-G when spec.backup / env is configured.
# If backups were not configured, you only have PVCs + replicas.
# Logical undo (lab):
kubectctl -n postgres-ha exec -it $(kubectctl -n postgres-ha get pod -l spilo-role=master -o name) -- \
    psql -U postgres -d appdb -c "\d+"
# Point-in-time: clone a new Postgresql CR with clone.spec (see operator docs)
# rather than rewinding production in place.
```

Resolve and verify. Prefer clone-from-cluster or clone-from-S3 into a new CR (e.g. acid-postgres-ha-clone) in the same namespace, validate, then switch the application Service. Never pg_resetwal on a running HA cluster.

Exercise 13 — WAL archive_command failing (WAL log failure)

Symptom. pg_stat_archiver.failed_count climbs. pg_wal on the PVC grows without bound. Logs: archive command failed with exit code 1 / AccessDenied / NoSuchBucket.

Diagnose.

```
MASTER=$(kubectctl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master \
    -o jsonpath='{.items[0].metadata.name}')
kubectctl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
    "SELECT archived_count, last_archived_wal, last_archived_time,
        failed_count, last_failed_wal, last_failed_time
    FROM pg_stat_archiver;"
kubectctl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW archive_command;"
kubectctl -n postgres-ha exec "$MASTER" -- ls /run/etc/wal-e.d/env
kubectctl -n postgres-ha exec "$MASTER" -- sh -c 'echo test-seg > /tmp/w; \
    envdir /run/etc/wal-e.d/env wal-g wal-push /tmp/w; echo exit:$?'
kubectctl -n postgres-ha exec "$MASTER" -- df -h /home/postgres/pgdata
kubectctl -n postgres-ha logs "$MASTER" --tail=200 | egrep -i 'archive|wal-g|wal-e|ERROR'
```

Resolve and verify. Fix credentials, endpoint, bucket name, path-style, or the IAM role on the node. Confirm USE_WALG_BACKUP=true so archive_command calls wal-g not wal-e. After a successful wal-push, SELECT pg_switch_wal() and watch last_archived_wal move. If pg_wal already filled the disk, free space (see Ex 7 / 16) BEFORE archive can resume — Postgres will not archive while the filesystem is 100% full.

Exercise 14 — WAL not recycled (replication slot holds segments)

Symptom. pg_wal uses tens of GB even though archive_command succeeds. Primary disk slowly fills.

Diagnose.

```
MASTER=$(kubectl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master \
-o jsonpath='{.items[0].metadata.name}')
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
"SELECT slot_name, slot_type, active, restart_lsn, wal_status,
pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS retained_bytes
FROM pg_replication_slots;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
"SELECT setting FROM pg_settings WHERE name IN
('max_slot_wal_keep_size', 'wal_keep_size', 'min_wal_size', 'max_wal_size');"
kubectl -n postgres-ha exec "$MASTER" -- du -sh /home/postgres/pgdata/pgroot/data/pg_wal
kubectl -n postgres-ha exec "$MASTER" -- patronictl list
```

Resolve and verify. An inactive physical slot (old replica name, leftover logical slot, Debezium) pins WAL. If the consumer is gone: `SELECT pg_drop_replication_slot('slot');` only after you confirm nothing still needs it. Set `max_slot_wal_keep_size` so a dead slot cannot take the disk. Do not delete `pg_wal` files by hand.

Exercise 15 — WAL corruption / missing segment on a replica

Symptom. Replica logs: requested WAL segment 000000010000000000000000AB has already been removed or invalid magic number. Replica state = start failed / crashed.

Diagnose.

```
kubectl -n postgres-ha logs acid-postgres-ha-2 --tail=200 | egrep -i 'WAL|wal|timeline|rewind|fatal'
kubectl -n postgres-ha exec acid-postgres-ha-0 -- patronictl list
# On the primary, confirm the segment still exists locally or in the bucket
kubectl -n postgres-ha exec acid-postgres-ha-0 -- \
ls /home/postgres/pgdata/pgroot/data/pg_wal | tail
kubectl -n postgres-ha exec acid-postgres-ha-0 -- \
envdir /run/etc/wal-e.d/env wal-g wal-fetch 000000010000000000000000AB /tmp/seg ; echo $?
```

Resolve and verify. If the primary already recycled the WAL and the replica cannot fetch it from the archive, the replica cannot catch up. `patronictl reinit` the member (Ex 8) so it takes a fresh basebackup. If `wal-fetch` itself fails, fix `restore_command` / `bucket` first or `reinit` will also fail when it tries to apply WAL after backup.

Exercise 16 — How data is stored: PVC full, wrong mount, or data on emptyDir

Symptom. After a node drain, a pod comes back empty or “new cluster” / `initdb`. Or `df` shows overlay filesystem instead of the PVC.

Diagnose.

```
kubectl -n postgres-ha get pvc -l cluster-name=acid-postgres-ha
kubectl -n postgres-ha get pod acid-postgres-ha-0 -o jsonpath='{.spec.volumes}' | jq .
kubectl -n postgres-ha exec acid-postgres-ha-0 -- df -h /home/postgres/pgdata
kubectl -n postgres-ha exec acid-postgres-ha-0 -- \
ls -la /home/postgres/pgdata/pgroot/data | head
# Who owns PGDATA?
kubectl -n postgres-ha exec acid-postgres-ha-0 -- id
kubectl -n postgres-ha exec acid-postgres-ha-0 -- \
stat -c '%U %G %a' /home/postgres/pgdata/pgroot/data
```

Resolve and verify. Spilo expects the named PVC `pgdata--`. If the volume template was lost or the pod was run without the operator, data sits on ephemeral storage and vanishes on restart. Recreate via the PostgreSQL CR only. Ownership must be `postgres` (uid 101 / gid 103 in current Spilo). After `resize`: patch `spec.volume.size` upward (StorageClass `allowVolumeExpansion: true`), wait for PVC capacity to change, then `df -h` inside the pod.

Exercise 17 — Base backup never appears in the bucket

Symptom. wal-g backup-list is empty hours after the cluster is Running. Restore tests fail.

Diagnose.

```
MASTER=$(kubectl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master \
-o jsonpath='{.items[0].metadata.name}')
kubectl -n postgres-ha exec "$MASTER" -- env | egrep 'WALG|WALE|WAL_S3|USE_WALG|BACKUP_'
kubectl -n postgres-ha exec "$MASTER" -- ls -l /run/etc/wal-e.d/env
kubectl -n postgres-ha exec "$MASTER" -- \
  envdir /run/etc/wal-e.d/env /scripts/postgres_backup.sh /home/postgres/pgdata/pgroot/data
kubectl -n postgres-ha exec "$MASTER" -- envdir /run/etc/wal-e.d/env wal-g backup-list
# Cron inside the pod (Spilo backup schedule)
kubectl -n postgres-ha exec "$MASTER" -- crontab -l 2>/dev/null || true
kubectl -n postgres-ha logs "$MASTER" --tail=200 | egrep -i 'backup|wal-g'
```

Resolve and verify. Empty list means the nightly job never ran or cannot write. Run postgres_backup.sh by hand and read stderr. Typical causes: USE_WALG_BACKUP=false (still calling wal-e), wrong WALG_S3_PREFIX, clock/IAM, or the script refuses because a backup is already in progress. BACKUP_NUM_TO_RETAIN too low plus a failed run can delete the only good backup — list before you retain-prune.

Exercise 18 — Backup succeeds but restore/clone cannot see it

Symptom. Clone pod: Clone failed / wal-g backup-list returned empty / using wal-e instead of wal-g.

Diagnose.

```
# Compare LIVE env vs CLONE env inside the clone pod
kubectl -n postgres-ha logs acid-postgres-ha-clone-0 --tail=250
kubectl -n postgres-ha exec acid-postgres-ha-clone-0 -- ls /run/etc/wal-e.d/
kubectl -n postgres-ha exec acid-postgres-ha-clone-0 -- \
  sh -c 'echo LIVE; envdir /run/etc/wal-e.d/env printenv | egrep WALG\|WAL_; \
  echo CLONE; envdir /run/etc/wal-e.d/env-clone-acid-postgres-ha printenv | egrep WALG\|WAL_|CLONE'
# Source cluster uid MUST match the path WAL-G used when the backup was taken
kubectl -n postgres-ha get postgresql acid-postgres-ha -o jsonpath='{.metadata.uid}'; echo
kubectl -n postgres-ha get postgresql acid-postgres-ha-clone -o yaml | sed -n '/clone:/,/patroni:/p'
```

Resolve and verify. Clone uses CLONE_* variables, not the live WAL_* ones. Set CLONE_USE_WALG_RESTORE=true, CLONE_WAL_S3_BUCKET, CLONE_AWS_*, and the source uid if your prefix includes it. A future timestamp means “latest backup”; a timestamp before the first basebackup fails. Do not clone into a bucket path that the new cluster will also archive to without a distinct prefix — cloned WAL can poison the source history files.

Exercise 19 — PITR restore lands on the wrong time or incomplete WAL chain

Symptom. Clone comes up but data is older than the requested timestamp, or recovery stops with “requested recovery stop point is before consistent recovery point”.

Diagnose.

```
# You need: a basebackup taken BEFORE the target time, and every WAL after that backup
# up to the target time already pushed by archive_command.
# Inject in lab: take backup, write rows, pg_switch_wal, wait for archive, then clone
# with timestamp between write and “now”.
kubectl -n postgres-ha exec "$MASTER" -- envdir /run/etc/wal-e.d/env wal-g backup-list
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
  "SELECT last_archived_wal, last_archived_time FROM pg_stat_archiver;"
# After clone is Running
kubectl -n postgres-ha exec acid-postgres-ha-clone-0 -- \
  psql -U postgres -d appdb -c "SELECT max(ts) FROM failover_lab;"
```

Resolve and verify. PITR cannot invent WAL that was never archived. If archive_command was broken during the window (Ex 13), that window is unrestoreable. Pick a timestamp after backup-list’s newest backup that finished AND after last_archived_time covering your writes. Verify with a row you inserted at a known time. Keep source cluster untouched until the clone is validated.

Exercise 20 — Streaming replication broken (state != streaming)

Symptom. `pg_stat_replication` empty or state = catchup / startup for many minutes. `patronictl lag` column grows. Read Service returns stale data.

Diagnose.

```
MASTER=$(kubectl -n postgres-ha get pod -l cluster-name=acid-postgres-ha,spilo-role=master \
-o jsonpath='{.items[0].metadata.name}')
kubectl -n postgres-ha exec "$MASTER" -- patronictl list
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -x -c \
"SELECT * FROM pg_stat_replication;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
"SELECT pid, username, application_name, client_addr, state, sent_lsn, replay_lsn,
pg_wal_lsn_diff(sent_lsn, replay_lsn) lag
FROM pg_stat_replication;"
# From a replica
kubectl -n postgres-ha exec acid-postgres-ha-2 -- psql -U postgres -c \
"SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();"
kubectl -n postgres-ha exec acid-postgres-ha-2 -- \
curl -s localhost:8008/patroni | jq '{state,role,replication}'
# Network path primary:5432
kubectl -n postgres-ha exec acid-postgres-ha-2 -- \
getent hosts acid-postgres-ha-0.acid-postgres-ha.postgres-ha.svc
```

Resolve and verify. Empty `pg_stat_replication` means the replica is not connected (auth, `pg_hba`, DNS, NetworkPolicy, replica in rewind). Catchup that never ends is usually disk saturation on the replica or a huge lag after the primary ran solo. Compare LSNs. If lag > `maximum_lag_on_failover` the replica is ineligible for failover. Repair: fix network/auth; if WAL needed is gone, `reinit` (Ex 8 / 15).

Exercise 21 — Replica is streaming but writes still fail after failover

Symptom. `patronictl` shows a new Leader, but apps using the master Service get read-only transaction / recovery mode, or the Service still points at the old pod.

Diagnose.

```
kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha --show-labels
kubectl -n postgres-ha get ep acid-postgres-ha -o yaml
kubectl -n postgres-ha get ep acid-postgres-ha-repl -o yaml
# Patroni REST vs Service
for p in 0 1 2; do
  echo -n "pod $p role: "
  kubectl -n postgres-ha exec acid-postgres-ha-$p -- curl -s localhost:8008/primary ; echo
done
# App path
psql "host=127.0.0.1 port=15432 user=postgres dbname=appdb sslmode=require" \
-c "SELECT pg_is_in_recovery(), inet_server_addr();"

```

Resolve and verify. Two planes must agree: Patroni leader lock AND Kubernetes Endpoints for `acid-postgres-ha`. If REST says primary but Endpoints still list the old IP, check the operator and the pod ServiceAccount's right to patch Endpoints. Clients must use the Service DNS, drop pooled connections on failover (PgBouncer `server_reset_query` / pool recycle). `ssl` + hostname verification can pin an old cert.

Exercise 22 — Synchronous streaming: commit hangs

Symptom. INSERT waits for tens of seconds or until `statement_timeout`. `synchronous_commit` is on and `synchronous_standby_names` is not empty.

Diagnose.

```
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW synchronous_commit;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SHOW synchronous_standby_names;"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c \
    "SELECT application_name, sync_state, state FROM pg_stat_replication;"
# CR equivalent
kubectl -n postgres-ha get postgresql acid-postgres-ha -o yaml | sed -n '/patroni:\/,/enable/p'
```

Resolve and verify. With `spec.patroni.synchronous_mode: true` the primary will not commit until a sync replica flushes WAL. If that replica dies, commits stall until Patroni removes it from `synchronous_standby_names` (ttl/loop_wait). For a lab, disable `synchronous_mode` or keep at least two healthy replicas. Check `sync_state` is sync on one member and streaming.

Exercise 23 — Restore test drill (prove backups, do not assume)

Symptom. You have backups listed but have never started a second cluster from them.

Diagnose.

```
# 1. Write a marker row and switch WAL so it is archived
psql ... -c "CREATE TABLE IF NOT EXISTS restore_marker(ts timestamptz default now(), note text);"
psql ... -c "INSERT INTO restore_marker(note) VALUES ('before-backup');"
kubectl -n postgres-ha exec "$MASTER" -- \
    envdir /run/etc/wal-e.d/env /scripts/postgres_backup.sh /home/postgres/pgdata/pgroot/data
psql ... -c "INSERT INTO restore_marker(note) VALUES ('after-backup-should-need-wal');"
kubectl -n postgres-ha exec "$MASTER" -- psql -U postgres -c "SELECT pg_switch_wal();"
sleep 15
kubectl -n postgres-ha exec "$MASTER" -- envdir /run/etc/wal-e.d/env wal-g backup-list

# 2. Apply acid-postgres-ha-clone with spec.clone.cluster + timestamp AFTER the second INSERT
# 3. When clone Running:
psql-to-clone -c "SELECT * FROM restore_marker ORDER BY ts;"
# Expect BOTH rows if PITR timestamp is after the second insert and WAL archived.
# Expect ONLY the first row if you cloned from the backup with no WAL apply window.
```

Resolve and verify. A backup you have never restored is not a backup. Schedule this drill on a clone CR in `postgres-ha`, then delete the clone. Record `backup-list` output, timestamp used, and row counts. That sheet is the acceptance test for Ex 17–19.

8. Health-check checklist (run after every change)

```

NS=postgres-ha
CL=acid-postgres-ha

kubectl -n $NS get postgresql $CL
kubectl -n $NS get pods -l cluster-name=$CL -o wide --show-labels
kubectl -n $NS get svc,ep -l cluster-name=$CL
kubectl -n $NS exec $(NS_MASTER) -- patronictl list # define NS_MASTER similarly

# Exactly one master label
kubectl -n $NS get pods -l cluster-name=$CL,spilo-role=master --no-headers | wc -l # expect 1

# Replication streaming
# On master: SELECT count(*) FROM pg_stat_replication WHERE state = 'streaming';
# Expect numberOfInstances-1

# Writes through the Service
psql "host=$CL.$NS.svc user=postgres dbname=appdb sslmode=require" -c "SELECT now();"

# Operator is up
kubectl -n postgres-operator get pods

# WAL archive is moving
# SELECT last_archived_wal, last_failed_wal, failed_count FROM pg_stat_archiver;

# Streaming
# SELECT application_name, state, sync_state, pg_wal_lsn_diff(sent_lsn, replay_lsn) FROM pg_stat_replication;

# Backups exist
# envdir /run/etc/wal-e.d/env wal-g backup-list

```

9. Command and object cheat sheet

Need	Command
Cluster CR	<code>kubectl -n postgres-ha get postgresql acid-postgres-ha -o yaml</code>
Pods + roles	<code>kubectl -n postgres-ha get pods -l cluster-name=acid-postgres-ha --show-labels</code>
Patroni list	<code>kubectl -n postgres-ha exec acid-postgres-ha-0 -- patronictl list</code>
Switchover	<code>patronictl switchover --candidate acid-postgres-ha-1 --force</code>
Failover (last resort)	<code>patronictl failover --candidate <member> --force</code>
Reinit replica	<code>patronictl reinit acid-postgres-ha <member> --force</code>
Restart Postgres via Patroni	<code>patronictl restart acid-postgres-ha <member></code>
Reload config	<code>patronictl reload acid-postgres-ha</code>
Master secret	<code>postgres.acid-postgres-ha.credentials.postgresql.acid.zalan.do</code>
Operator logs	<code>kubectl -n postgres-operator logs -l name=postgres-operator</code>
Edit live CR	<code>kubectl -n postgres-ha edit postgresql acid-postgres-ha</code>
Delete cluster (DESTROYS PVCs if reclaim policy says so)	<code>kubectl -n postgres-ha delete postgresql acid-postgres-ha</code>
WAL archive stats	<code>SELECT * FROM pg_stat_archiver;</code>
Force WAL archive	<code>SELECT pg_switch_wal();</code>
List WAL-G backups	<code>envdir /run/etc/wal-e.d/env wal-g backup-list</code>
Take base backup now	<code>envdir /run/etc/wal-e.d/env /scripts/postgres_backup.sh \$PGDATA</code>
Fetch one WAL	<code>envdir /run/etc/wal-e.d/env wal-g wal-fetch <seg> /tmp/seg</code>
Replication view	<code>SELECT * FROM pg_stat_replication;</code>
Slots holding WAL	<code>SELECT * FROM pg_replication_slots;</code>

Need	Command
PVC usage	<code>df -h /home/postgres/pgdata</code>

10. Common Patroni states

State / role	Meaning	Typical action
Leader / running	Primary, lock held	None
Replica / streaming	Healthy standby	None
Replica / creating replica	basebackup in progress	Wait; check disk and logs
stopped	Postgres down, Patroni up	logs; start or reinit
start failed	Postgres will not start	PGDATA / parameters / disk
crashed	Unexpected exit	logs; may auto-restart
Timeline history mismatch	Diverged after failover	reinit that replica
unknown / DCS fail	Cannot talk to K8s API or etcd	RBAC, DNS, NetworkPolicy

11. Cleanup

```
# Kubernetes lab
kubectl -n postgres-ha delete postgresql acid-postgres-ha
# Wait until pods and PVCs are gone (check operator config delete protection)
kubectl -n postgres-ha get pvc,pods
kubectl delete ns postgres-ha
helm uninstall postgres-operator -n postgres-operator
kubectl delete ns postgres-operator

# Docker lab
cd patroni && docker compose down -v
```

12. Suggested lab scoring rubric

Task	Points	Pass criteria
Docker cluster up, patronictl list healthy	10	1 leader, 2 streaming
Docker failover by stopping leader	10	Writes via :5000 succeed after election
Operator + CR in postgres-ha Running	15	3 Ready pods, STATUS Running
Connect via master Service + Secret	10	psql SELECT 1 works
Planned switchover	10	Endpoints follow new leader
Exercise 5 unplanned kill	15	New leader + old pod rejoins streaming
Exercise 8 reinit replica	10	Member returns to streaming
Exercise 9 or 10 completed with notes	10	Written diagnosis
Checklist all green	5	One master, streaming count correct
WAL archive healthy (Ex 13)	5	failed_count stable, last_archived moves
backup-list non-empty (Ex 17)	5	Manual postgres_backup.sh + list
Clone / PITR drill (Ex 23)	10	restore_marker rows match chosen timestamp

13. Notes and references

Zalando Postgres Operator: <https://github.com/zalando/postgres-operator> — quickstart, administrator guide, and reference for the postgresql CRD (users, databases, clone, enableLogicalBackup, connection pooler, major version upgrade).

Patroni: <https://patroni.readthedocs.io> — REST API, patronictl, DCS, rewind, synchronous_mode, pause (maintenance mode: patronictl pause / resume).

Spilo (the image the operator runs): PostgreSQL + Patroni + WAL-G. Environment variables SPILO_* and PATRONI_* are generated from the CR; do not fight them by hand-editing postgres.yml inside a live pod — the next reconcile will overwrite it.

Maintenance mode

Before a risky host or kubelet action: patronictl pause so Patroni will not failover mid-change. patronictl resume afterwards. Leaving pause on is a common reason “failover did not happen”.

Document generated as a lab workbook. Commands assume Linux amd64/arm64, kubectl context already authenticated, and that you substituted StorageClass and resource sizes for your cluster. PostgreSQL 16 and operator v1.13+ / v2.x are compatible with the manifests shown; always confirm supported version pairs in the operator release notes before upgrading production.